EXPLORATORY ANALYSIS ON IMDB TOP MOVIES

Checkpoint Challenge 1: Analyzing and Visualizing Insights from the IMDB Movie Dataset

Performing exploratory data analysis (EDA) on IMDB's top movies dataset can provide valuable insights into the characteristics and trends of highly rated films

```
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px


# Loading the dataset

import pandas as pd

df = pd.read_csv(
    "MOVIE ANALYSIS GG/upload_c99d6907-13d2-433e-b608-
3cb9d079ccd9.csv",
    sep=None,
    engine="python",
    encoding="utf-8-sig"
)

# Display first 10 rows
df.head(10)
```

```
  Unnamed: 0      id                                title release_date
\
0           0  385687                                Fast X   2023-05-17

1           1  603692                 John Wick: Chapter 4   2023-03-22

2           2  502356          The Super Mario Bros. Movie   2023-04-05

3           3  569094  Spider-Man: Across the Spider-Verse   2023-05-31

4           4  536437                             Hypnotic   2023-05-11

5           5  667538     Transformers: Rise of the Beasts   2023-06-06

6           6  890771                      The Black Demon   2023-04-26

7           7  447277                   The Little Mermaid   2023-05-18

8           8   76600             Avatar: The Way of Water   2022-12-14
```

```
9          9  713704                          Evil Dead Rise   2023-04-12


                                                 genres original_language
\
0                    ['Action', 'Crime', 'Thriller']            English

1                    ['Action', 'Thriller', 'Crime']            English

2  ['Animation', 'Family', 'Adventure', 'Fantasy'...            English

3  ['Action', 'Adventure', 'Animation', 'Science ...            English

4         ['Mystery', 'Thriller', 'Science Fiction']            English

5         ['Action', 'Adventure', 'Science Fiction']            English

6                              ['Horror', 'Thriller']            English

7      ['Adventure', 'Family', 'Fantasy', 'Romance']            English

8         ['Science Fiction', 'Adventure', 'Action']            English

9                              ['Thriller', 'Horror']            English


   vote_average  vote_count  popularity  \
0           7.4      1347.0    8363.473
1           7.9      2896.0    4210.313
2           7.8      4628.0    3394.458
3           8.8      1160.0    2859.047
4           6.5       154.0    2654.854
5           7.4       195.0    2453.807
6           6.5       151.0    1777.200
7           6.2       616.0    1448.640
8           7.7      8493.0    1344.884
9           7.1      1597.0    1285.781

                                             overview       budget  \
0  Over many missions and against impossible odds...  340000000.0
1  With the price on his head ever increasing, Jo...   90000000.0
2  While working underground to fix a water main,...  100000000.0
3  After reuniting with Gwen Stacy, Brooklyn’s fu...  100000000.0
4  A detective becomes entangled in a mystery inv...   70000000.0
5  When a new threat capable of destroying the en...  200000000.0
6  Oilman Paul Sturges' idyllic family vacation t...          0.0
7  The youngest of King Triton’s daughters, and t...  250000000.0
8  Set more than a decade after the events of the...  460000000.0
9  Three siblings find an ancient vinyl that give...   15000000.0

                                production_companies       revenue
```

```
runtime  \
0  ['Universal Pictures', 'Original Film', 'One R...  6.520000e+08
142.0
1  ['Thunder Road', '87Eleven', 'Summit Entertain...  4.317692e+08
170.0
2  ['Universal Pictures', 'Illumination', 'Ninten...  1.308767e+09
92.0
3  ['Columbia Pictures', 'Sony Pictures Animation...  3.135222e+08
140.0
4  ['Studio 8', 'Solstice Productions', 'Ingeniou...  0.000000e+00
94.0
5  ['Skydance', 'Paramount', 'di Bonaventura Pict...  1.710455e+08
127.0
6  ['Mucho Mas Media', 'Silk Mass', 'The Avenue E...  0.000000e+00
100.0
7  ['Walt Disney Pictures', 'Lucamar Productions'...  4.140000e+08
135.0
8  ['20th Century Studios', 'Lightstorm Entertain...  2.320250e+09
192.0
9  ['New Line Cinema', 'Ghost House Pictures', 'R...  1.415121e+08
96.0

                                              tagline
0                        The end of the road begins.
1                          No way back, one way out.
2                                                NaN
3            It's how you wear the mask that matters
4                             Control is an illusion.
5                                      Unite or fall.
6                                  Nature bites back.
7  Watch and you'll see, some day I'll be, part o...
8                                  Return to Pandora.
9                            Mommy loves you to death.
```

##Exploring the Dataset

```
df.shape
```

```
(10001, 15)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10001 entries, 0 to 10000
Data columns (total 15 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Unnamed: 0            10001 non-null  object
 1   id                    10001 non-null  int64
 2   title                 10001 non-null  object
 3   release_date          9978 non-null   object
```

```
 4   genres                 10001 non-null  object
 5   original_language      10000 non-null  object
 6   vote_average           10000 non-null  float64
 7   vote_count             10000 non-null  float64
 8   popularity             10000 non-null  float64
 9   overview               9924 non-null   object
 10  budget                 9999 non-null   float64
 11  production_companies   9999 non-null   object
 12  revenue                9999 non-null   float64
 13  runtime                9999 non-null   float64
 14  tagline                7383 non-null   object
dtypes: float64(6), int64(1), object(8)
memory usage: 1.1+ MB

df.describe()

                  id  vote_average    vote_count    popularity
budget  \
count  1.000100e+04  10000.000000  10000.000000  10000.000000
9.999000e+03
mean   3.001465e+05      6.319090   1558.294800     33.542340
1.993800e+07
std    3.379901e+05      1.460605   2887.861217    126.495621
3.870582e+07
min    0.000000e+00      0.000000      0.000000     12.297000
0.000000e+00
25%    1.150400e+04      5.900000    143.000000     14.599000
0.000000e+00
50%    1.184060e+05      6.600000    519.000000     18.582500
1.250000e+06
75%    5.379150e+05      7.100000   1584.000000     28.014500
2.300000e+07
max    1.136631e+06     10.000000  33822.000000   8363.473000
4.600000e+08

            revenue      runtime
count  9.999000e+03  9999.000000
mean   6.084337e+07   100.809581
std    1.553073e+08    27.850165
min    0.000000e+00     0.000000
25%    0.000000e+00    90.000000
50%    2.133452e+06   100.000000
75%    5.106230e+07   115.000000
max    2.923706e+09   366.000000

df.columns

Index(['Unnamed: 0', 'id', 'title', 'release_date', 'genres',
       'original_language', 'vote_average', 'vote_count',
'popularity',
```

```
       'overview', 'budget', 'production_companies', 'revenue',
'runtime',
       'tagline'],
      dtype='object')
```

```
any(df.isnull().sum())
```

```
True
```

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
any(df.duplicated())
```

```
False
```

```
# removing the missing values
```

```
df.dropna(axis=0, inplace=True)
```

```
any(df.isna().sum())
```

```
False
```

```
df.shape
```

```
(7381, 15)
```

```
df.describe()
```

```
                  id  vote_average    vote_count   popularity
budget  \
count  7.381000e+03   7381.000000   7381.000000  7381.000000
7.381000e+03
mean   2.283510e+05      6.481927   1976.777266    34.159110
2.565858e+07
std    2.927978e+05      1.083383   3204.333042   137.731149
4.244034e+07
min    5.000000e+00      0.000000      0.000000    12.297000
0.000000e+00
25%    1.009000e+04      6.000000    291.000000    14.691000
0.000000e+00
50%    4.163000e+04      6.600000    816.000000    18.860000
8.000000e+06
75%    4.265430e+05      7.200000   2131.000000    28.188000
3.200000e+07
max    1.136434e+06     10.000000  33822.000000  8363.473000
4.600000e+08

            revenue      runtime
count  7.381000e+03  7381.000000
```

```
mean    7.898499e+07   105.654925
std     1.740644e+08    22.792406
min     0.000000e+00     0.000000
25%     0.000000e+00    93.000000
50%     1.477986e+07   103.000000
75%     8.054787e+07   117.000000
max     2.923706e+09   366.000000
```

```
#names of columns
```

```
df.columns
```

```
Index(['Unnamed: 0', 'id', 'title', 'release_date', 'genres',
       'original_language', 'vote_average', 'vote_count',
'popularity',
       'overview', 'budget', 'production_companies', 'revenue',
'runtime',
       'tagline'],
      dtype='object')
```

```
#dropping columns not in use
df.drop(['id','overview','tagline'],axis=1,inplace=True)
```

```
#Profiling data again to clear unusabale data and getting and getting
more accurate insight
```

```
df['original_language'].unique()
```

```
array(['English', 'Finnish', 'Polish', 'Korean', 'Japanese',
'Spanish',
       'Ukrainian', 'Russian', 'Chinese', 'Norwegian', 'Romanian',
       'French', 'German', 'Telugu', 'Icelandic', 'Italian',
'Swedish',
       'Macedonian', 'cn', 'Thai', 'Portuguese', 'Tagalog', 'Serbian',
       'Hindi', 'Vietnamese', 'Basque', 'Bulgarian', 'Galician',
       'Indonesian', 'Greek', 'Tamil', 'Danish', 'Dutch', 'Arabic',
       'Malayalam', 'Persian', 'Kannada', 'Bosnian', 'Norwegian
Bokmal',
       'Hungarian', 'Turkish', 'Dzongkha'], dtype=object)
```

```
#NO ROWS WITH XX AS LANGUANGE
```

```
#No. of movies with unique title
```

```
df['title'].nunique()
```

```
7133
```

```
#No. of movies with same title
df['title'].duplicated().sum()
```

```
np.int64(248)
```

```
# Remove duplicate rows
df = df.drop_duplicates()

(df==0).sum()
```

```
Unnamed: 0                0
title                     0
release_date              0
genres                    0
original_language         0
vote_average             72
vote_count               72
popularity                0
budget                 2480
production_companies      0
revenue                2288
runtime                  30
dtype: int64
```

```
#rows with 0 vote_avg or vote_count or runtime values

df[(df['vote_average']==0)| (df['vote_count']==0)|(df['runtime']==0)]
```

```
     Unnamed: 0                                      title
release_date  \
174         174                          Meg 2: The Trench   2023-08-
02
183         183                                     Barbie   2023-07-
19
265         264  Miraculous: Ladybug & Cat Noir, The Movie   2023-07-
05
296         295      Indiana Jones and the Dial of Destiny   2023-06-
28
317         316                    Insidious: The Red Door   2023-07-
05
...         ...                                       ...          ..
.
9146       9145                                    Deceit   2023-06-
15
9541       9540                                His Only Son   2023-03-
30
9744       9743                               Next Goal Wins   2023-11-
17
9753       9752                             The Color Purple   2023-12-
25
9797       9796            The Quest for Tom Sawyer's Gold   2023-06-
22

                                                  genres
original_language  \
```

```
174             ['Action', 'Science Fiction', 'Horror']
English
183                                          ['Comedy']
English
265   ['Animation', 'Fantasy', 'Action', 'Family', '...
French
296                              ['Adventure', 'Action']
English
317                                          ['Horror']
English
...                                                 ...
...
9146                                    ['Documentary']
Spanish
9541                                ['Drama', 'History']
English
9744                                 ['Comedy', 'Drama']
English
9753                       ['Drama', 'Romance', 'Music']
English
9797                              ['Adventure', 'Family']
English

      vote_average  vote_count  popularity       budget  \
174            0.0         0.0     146.569     130178.0
183            0.0         0.0     143.021  100000000.0
265            0.0         0.0     114.803   86000000.0
296            0.0         0.0     107.803  300000000.0
317            0.0         0.0     101.313          0.0
...            ...         ...         ...          ...
9146           0.0         0.0      12.990          0.0
9541           0.0         0.0      12.683     250000.0
9744           0.0         0.0      12.495          0.0
9753           0.0         0.0      12.490          0.0
9797           0.0         0.0      12.462          0.0

                                   production_companies     revenue
runtime
174   ['Apelles Entertainment', 'Warner Bros. Pictur...         0.0
116.0
183   ['LuckyChap Entertainment', 'Heyday Films', 'M...         0.0
104.0
265   ['ON Animation Studios', 'The Awakening Produc...         0.0
107.0
296                                  ['Lucasfilm Ltd.']         0.0
154.0
317          ['Blumhouse Productions', 'Stage 6 Films']         0.0
0.0
...                                                 ...         ...
```

```
...
9146                        ['Jump Cut', 'Sisyfos Film']         0.0
67.0
9541        ['RockBridge Productions', 'Angel Studios']  11480048.0
106.0
9744  ['The Imaginarium', 'Searchlight Pictures', 'F...         0.0
97.0
9753  ['Amblin Entertainment', 'SGS Pictures', 'Harp...         0.0
0.0
9797  ['Ace Entertainment Films', 'Grindstone Entert...         0.0
88.0

[80 rows x 12 columns]
```

```
#dropping rows with 0 values
df=df[~((df['vote_average']==0)| (df['vote_count']==0)|
(df['runtime']==0))]
```

```
# descriptive statics of the data frame
df.describe()
```

```
       vote_average    vote_count   popularity        budget
revenue  \
count   7301.000000   7301.000000  7301.000000  7.301000e+03
7.301000e+03
mean       6.545624   1998.405013    34.172792  2.574396e+07
7.984889e+07
std        0.874533   3215.137355   138.454154  4.226757e+07
1.748186e+08
min        1.000000      1.000000    12.297000  0.000000e+00
0.000000e+00
25%        6.000000    301.000000    14.686000  0.000000e+00
0.000000e+00
50%        6.600000    840.000000    18.825000  8.000000e+06
1.534000e+07
75%        7.200000   2161.000000    28.130000  3.200000e+07
8.210238e+07
max        8.800000  33822.000000  8363.473000  4.600000e+08
2.923706e+09

           runtime
count  7301.000000
mean    106.135735
std      21.598613
min       3.000000
25%      93.000000
50%     103.000000
75%     117.000000
max     366.000000
```

```
#adding profit column
df['profit']=df['revenue']-df['budget']

df.head()
```

```
  Unnamed: 0                                  title release_date  \
0          0                                Fast X   2023-05-17
1          1                 John Wick: Chapter 4   2023-03-22
3          3  Spider-Man: Across the Spider-Verse   2023-05-31
4          4                              Hypnotic   2023-05-11
5          5    Transformers: Rise of the Beasts   2023-06-06

                                              genres original_language
\
0                   ['Action', 'Crime', 'Thriller']           English

1                   ['Action', 'Thriller', 'Crime']           English

3  ['Action', 'Adventure', 'Animation', 'Science ...           English

4         ['Mystery', 'Thriller', 'Science Fiction']           English

5         ['Action', 'Adventure', 'Science Fiction']           English

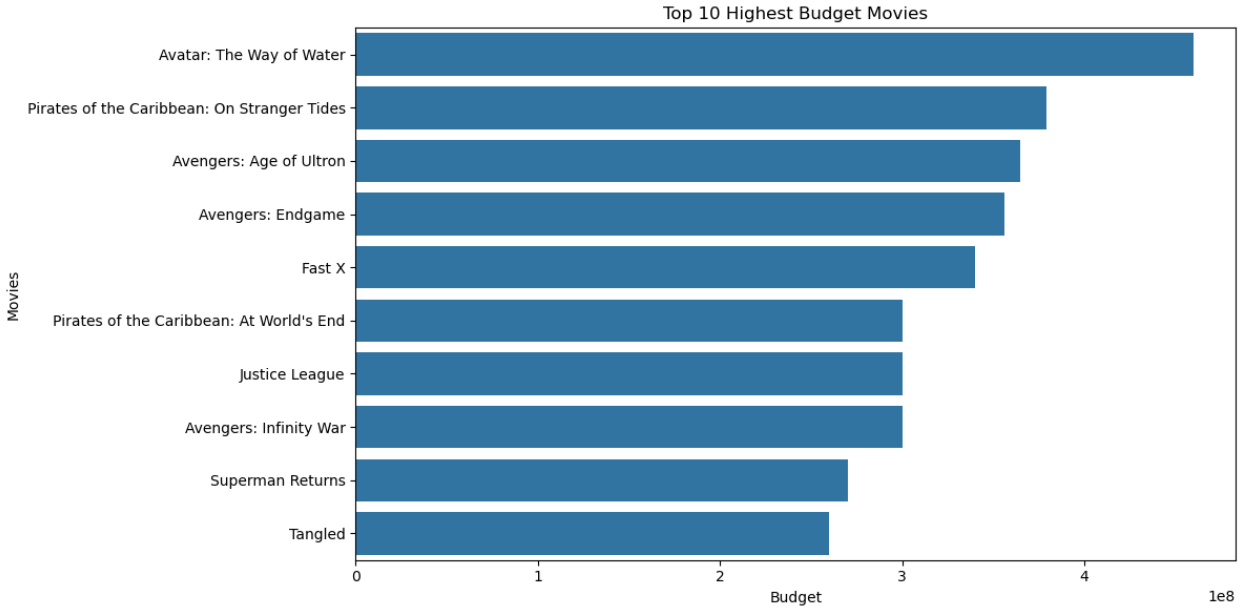

   vote_average  vote_count  popularity       budget  \
0           7.4      1347.0    8363.473  340000000.0
1           7.9      2896.0    4210.313   90000000.0
3           8.8      1160.0    2859.047  100000000.0
4           6.5       154.0    2654.854   70000000.0
5           7.4       195.0    2453.807  200000000.0

                                production_companies      revenue
runtime  \
0  ['Universal Pictures', 'Original Film', 'One R...  652000000.0
142.0
1  ['Thunder Road', '87Eleven', 'Summit Entertain...  431769198.0
170.0
3  ['Columbia Pictures', 'Sony Pictures Animation...  313522201.0
140.0
4  ['Studio 8', 'Solstice Productions', 'Ingeniou...          0.0
94.0
5  ['Skydance', 'Paramount', 'di Bonaventura Pict...  171045464.0
127.0

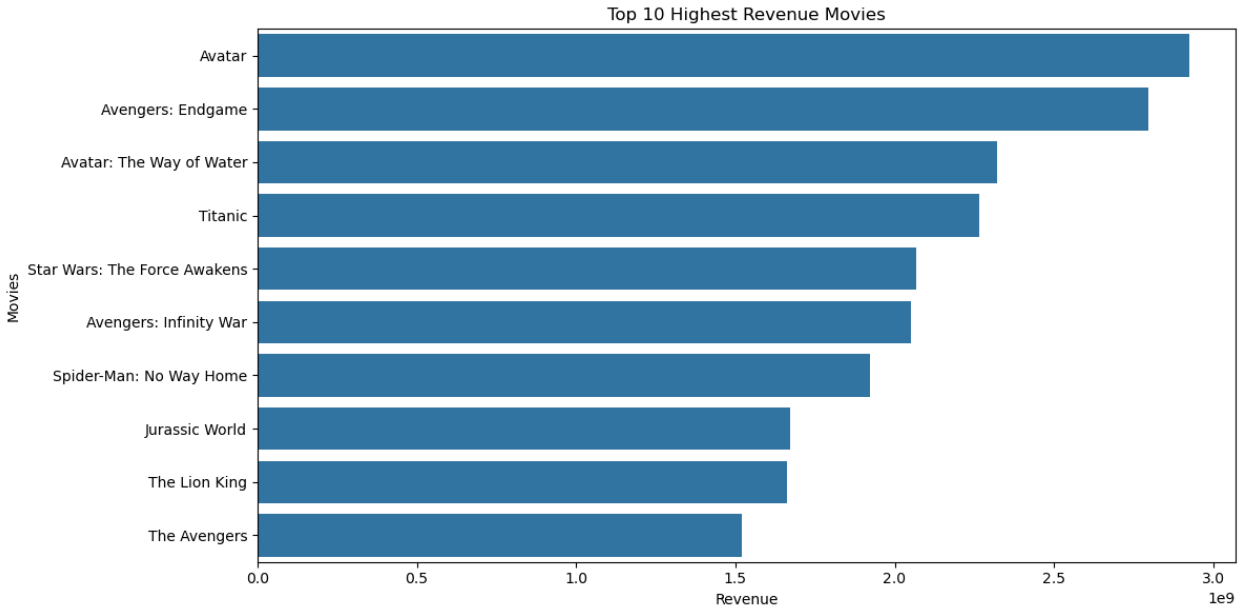
        profit
0  312000000.0
1  341769198.0
3  213522201.0
4  -70000000.0
5  -28954536.0
```

```
#saving our df as csv file
df.to_csv("movies.csv", index=False)

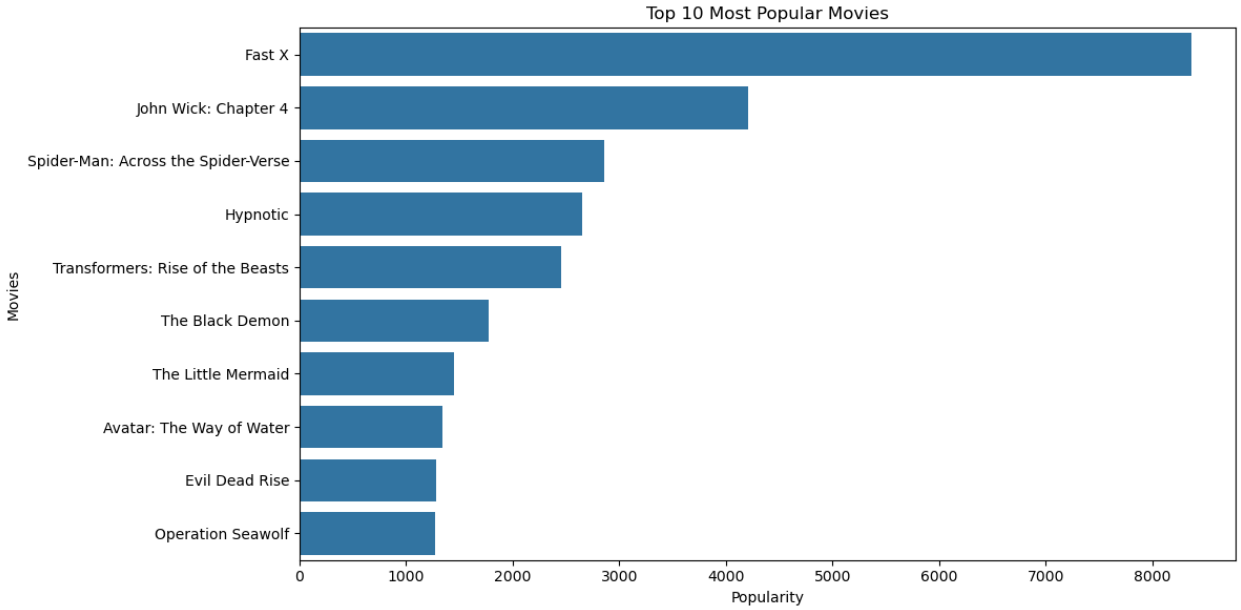
#  Top 10 movies by budget
max_budget = df.sort_values("budget", ascending=False).head(10)
plt.figure(figsize=(12, 6))
sns.barplot(data=max_budget, x="budget", y="title")
plt.title("Top 10 Highest Budget Movies")
plt.xlabel("Budget")
plt.ylabel("Movies")
plt.tight_layout()
plt.show()
```




```
#Top 10 movies by revenue
max_revenue = df.sort_values("revenue", ascending=False).head(10)
plt.figure(figsize=(12, 6))
sns.barplot(data=max_revenue, x="revenue", y="title")
plt.title("Top 10 Highest Revenue Movies")
plt.xlabel("Revenue")
plt.ylabel("Movies")
plt.tight_layout()
plt.show()
```




```
# Top 10 movies by popularity
max_popularity = df.sort_values("popularity",
ascending=False).head(10)
plt.figure(figsize=(12, 6))
sns.barplot(data=max_popularity, x="popularity", y="title")
plt.title("Top 10 Most Popular Movies")
plt.xlabel("Popularity")
plt.ylabel("Movies")
plt.tight_layout()
plt.show()
```



```
# Now we start to explore the data basically this is the story telling
and visuallizing part
max_budget = df.sort_values('budget', ascending=False).head(10)
#max_budget
max_revenue = df.sort_values('revenue', ascending=False).head(10)
#max_revenue
max_profit = df.sort_values('profit', ascending=False).head(10)
#max_profit
max_loss = df.sort_values('profit', ascending=True).head(10)
max_loss['loss'] = -max_loss['profit']
#max_loss
max_popularity = df.sort_values('popularity',
ascending=False).head(10)
#max_popularity
max_vc = df.sort_values('vote_count', ascending=False).head(10)
#max_vc
max_va = df.sort_values('vote_average', ascending=False).head(10)
#max va

fig, axs = plt.subplots(4,1,figsize=(10,28))

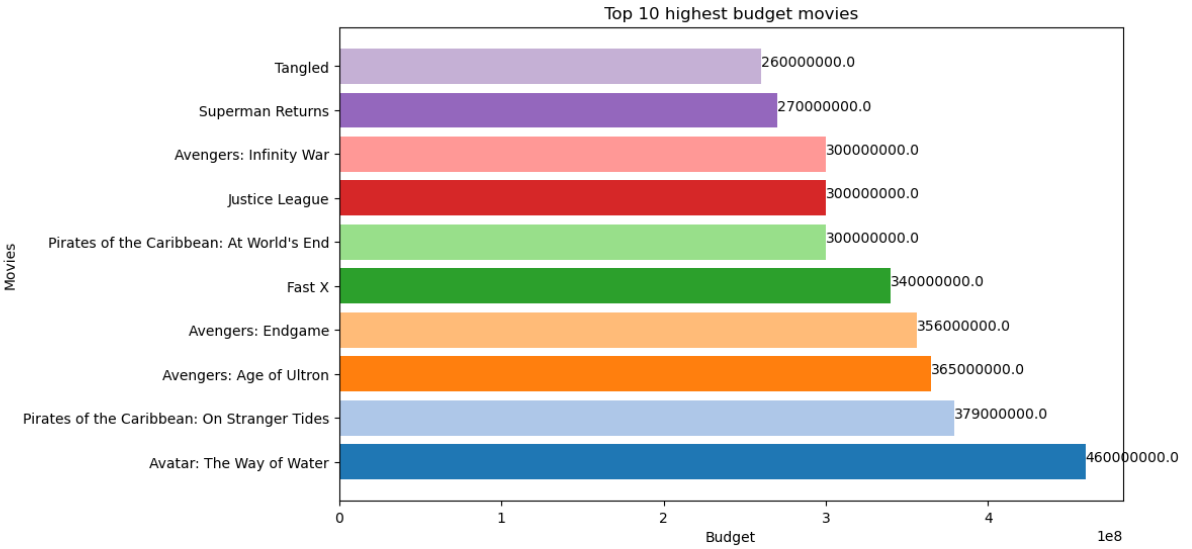
col_map = plt.get_cmap('tab20')
axs[0].barh(max_budget['title'], max_budget['budget'],
color=col_map.colors)
axs[0].set_title('Top 10 highest budget movies')
axs[0].set_xlabel('Budget')
axs[0].set_ylabel('Movies')
for i in range(len(max_budget)):
    axs[0].text(max_budget['budget'].iloc[i],
max_budget['title'].iloc[i], max_budget['budget'].iloc[i])

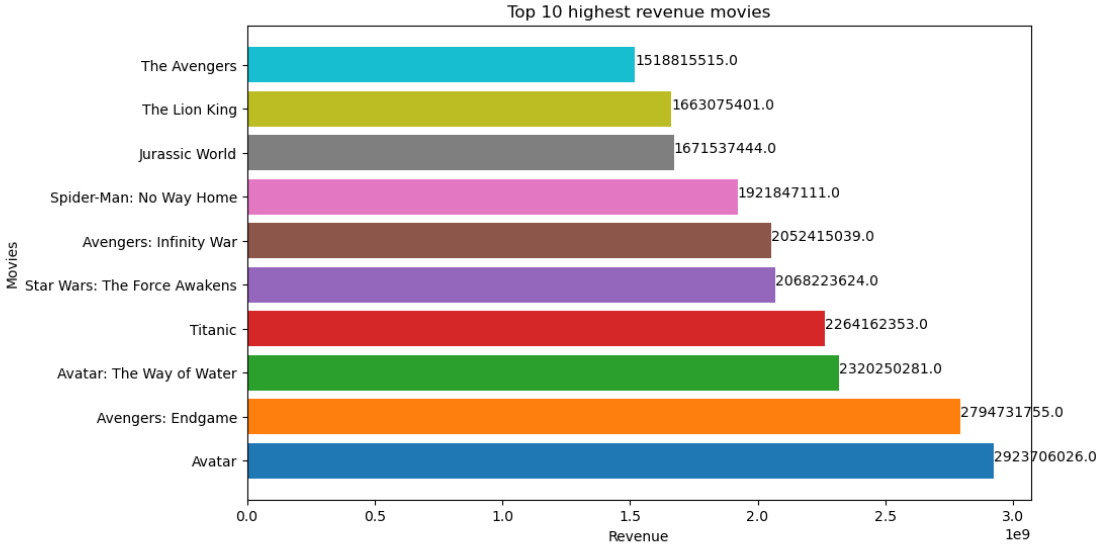
col_map = plt.get_cmap('tab10')
axs[1].barh(max_revenue['title'], max_revenue['revenue'],
color=col_map.colors)
axs[1].set_title('Top 10 highest revenue movies')
axs[1].set_xlabel('Revenue')
axs[1].set_ylabel('Movies')
for i in range(len(max_revenue)):
    axs[1].text(max_revenue['revenue'].iloc[i],
max_revenue['title'].iloc[i], max_revenue['revenue'].iloc[i])

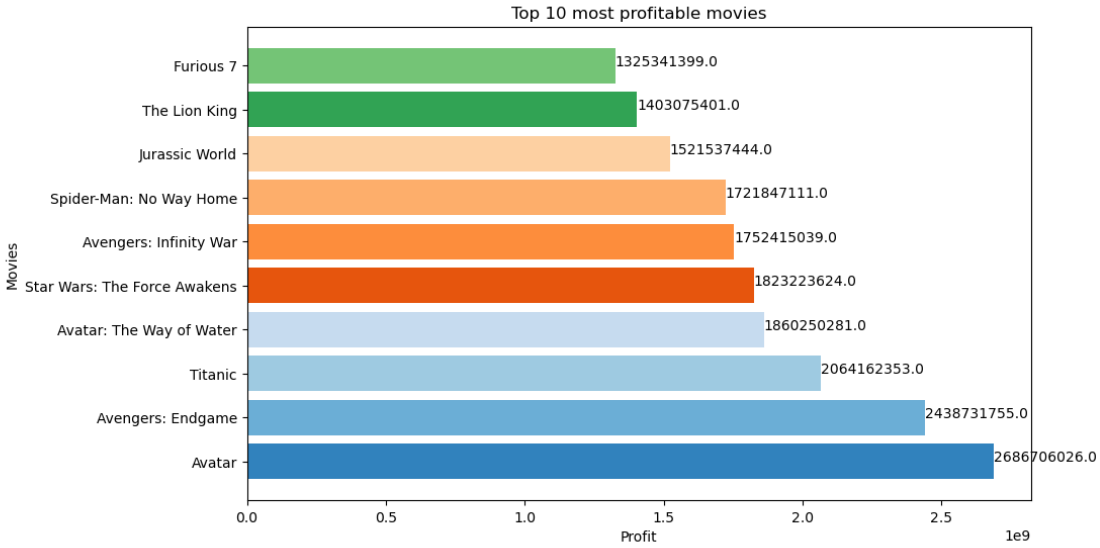
col_map = plt.get_cmap('tab20c')
axs[2].barh(max_profit['title'], max_profit['profit'],
color=col_map.colors)
axs[2].set_title('Top 10 most profitable movies')
axs[2].set_xlabel('Profit')
axs[2].set_ylabel('Movies')
for i in range(len(max_profit)):
    axs[2].text(max_profit['profit'].iloc[i],
max_profit['title'].iloc[i], max_profit['profit'].iloc[i])
```

```
col_map = plt.get_cmap('tab20')
axs[3].barh(max_loss['title'], max_loss['loss'], color=col_map.colors)
axs[3].set_title('Top 10 most loss making movies')
axs[3].set_xlabel('Loss')
axs[3].set_ylabel('Movies')
for i in range(len(max_loss)):
    axs[3].text(max_loss['loss'].iloc[i], max_loss['title'].iloc[i],
max_loss['loss'].iloc[i])

plt.show()
```







Top 10 most loss making movies

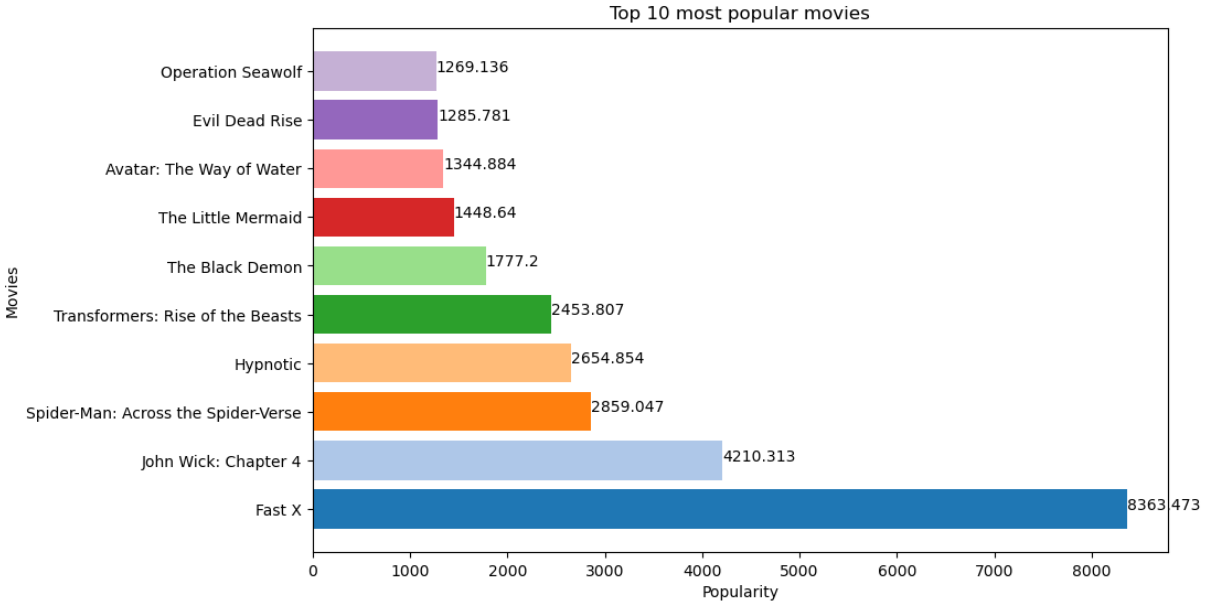
```
fig, axs = plt.subplots(3,1,figsize=(10,21))

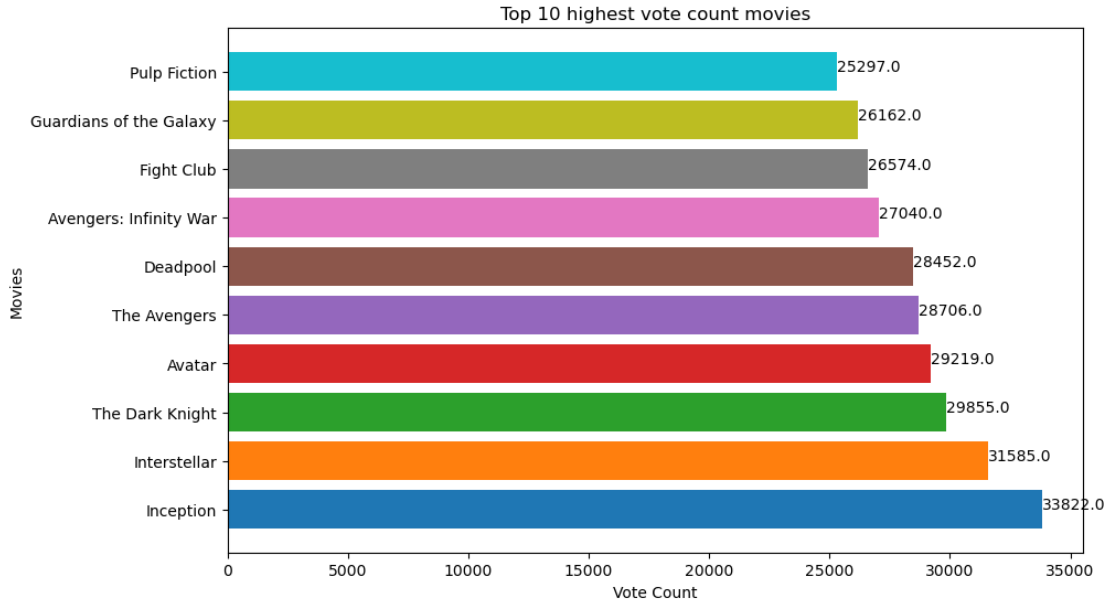
col_map = plt.get_cmap('tab20')
axs[0].barh(max_popularity['title'], max_popularity['popularity'],
color=col_map.colors)
axs[0].set_title('Top 10 most popular movies')
axs[0].set_xlabel('Popularity')
axs[0].set_ylabel('Movies')
for i in range(len(max_popularity)):
    axs[0].text(max_popularity['popularity'].iloc[i],
max_popularity['title'].iloc[i], max_popularity['popularity'].iloc[i])

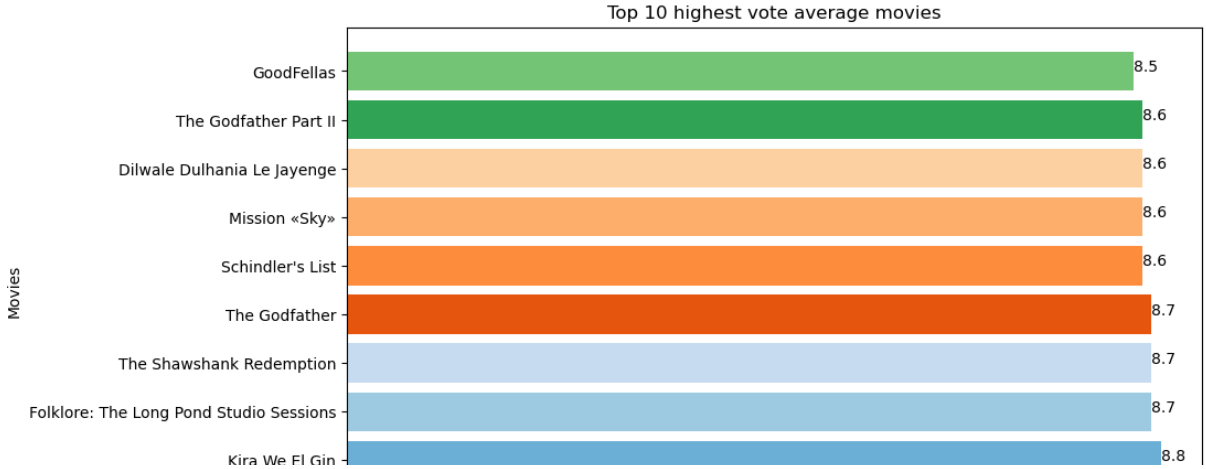
col_map = plt.get_cmap('tab10')
axs[1].barh(max_vc['title'], max_vc['vote_count'],
color=col_map.colors)
axs[1].set_title('Top 10 highest vote count movies')
axs[1].set_xlabel('Vote Count')
axs[1].set_ylabel('Movies')
for i in range(len(max_vc)):
    axs[1].text(max_vc['vote_count'].iloc[i], max_vc['title'].iloc[i],
max_vc['vote_count'].iloc[i])

col_map = plt.get_cmap('tab20c')
axs[2].barh(max_va['title'], max_va['vote_average'],
color=col_map.colors)
axs[2].set_title('Top 10 highest vote average movies')
axs[2].set_xlabel('Vote Average')
axs[2].set_ylabel('Movies')
for i in range(len(max_va)):
    axs[2].text(max_va['vote_average'].iloc[i],
max_va['title'].iloc[i], max_va['vote_average'].iloc[i])

plt.show()
```


Top 10 most popular movies
Operation Seawolf
Evil Dead Rise
Avatar: The Way of Water
The Little Mermaid
The Black Demon
Transformers: Rise of the Beasts
Hypnotic
Spider-Man: Across the Spider-Verse
John Wick: Chapter 4
Fast X
1269.136
1285.781
1344.884
1448.64
1777.2
2453.807
2654.854
2859.047
4210.313
8363.473
Movies
Popularity
0
1000
2000
3000
4000
5000
6000
7000
8000


Top 10 highest vote count movies
Pulp Fiction
Guardians of the Galaxy
Fight Club
Avengers: Infinity War
Deadpool
The Avengers
Avatar
The Dark Knight
Interstellar
Inception
25297.0
26162.0
26574.0
27040.0
28452.0
28706.0
29219.0
29855.0
31585.0
33822.0
Movies
Vote Count
0
5000
10000
15000
20000
25000
30000
35000


Top 10 highest vote average movies
GoodFellas
The Godfather Part II
Dilwale Dulhania Le Jayenge
Mission «Sky»
Schindler's List
The Godfather
The Shawshank Redemption
Folklore: The Long Pond Studio Sessions
Kira We El Gin
8.5
8.6
8.6
8.6
8.6
8.7
8.7
8.7
8.8
Movies

```
# : Number of unique genres
# The genres column contains string representations of lists.
import ast

unique_genres = set()
for genre_list in df["genres"].dropna():
    try:
        parsed_genres = ast.literal_eval(genre_list) if
isinstance(genre_list, str) else genre_list
        for genre in parsed_genres:
            if isinstance(genre, str):
                unique_genres.add(genre)
            elif isinstance(genre, dict) and "name" in genre:
                unique_genres.add(genre["name"])
    except (ValueError, SyntaxError, TypeError):
        continue

len(unique_genres)
```

```
19
```

```
# Top 5 movies in every genre
for genre in sorted(unique_genres):
    genre_movies = df[df["genres"].astype(str).str.contains(genre,
case=False, na=False)]
    top_movies = genre_movies.sort_values("popularity",
ascending=False).head(5)
    print("\nGenre:", genre)
    print(top_movies["title"].to_string(index=False))
```

```

Genre: Action
                              Fast X
               John Wick: Chapter 4
Spider-Man: Across the Spider-Verse
   Transformers: Rise of the Beasts
           Avatar: The Way of Water

Genre: Adventure
Spider-Man: Across the Spider-Verse
   Transformers: Rise of the Beasts
                 The Little Mermaid
           Avatar: The Way of Water
                  Operation Seawolf

Genre: Animation
               Spider-Man: Across the Spider-Verse
                 Spider-Man: Into the Spider-Verse
                      Puss in Boots: The Last Wish
Sword Art Online the Movie -Progressive- Scherz...
```

```
                                                  Suzume

Genre: Comedy
         Accident Man: Hitman's Holiday
               Shazam! Fury of the Gods
           Puss in Boots: The Last Wish
                                 Ghosted
Dungeons & Dragons: Honor Among Thieves

Genre: Crime
                          Fast X
          John Wick: Chapter 4
Accident Man: Hitman's Holiday
             One Day as a Lion
                           Snag

Genre: Documentary
               The Biggest Little Farm: The Return
Harry Potter 20th Anniversary: Return to Hogwarts
                                After Porn Ends 2
                                        Bama Rush
                                 Hot Girls Wanted

Genre: Drama
Operation Seawolf
     Mother's Day
        Creed III
           Suzume
Top Gun: Maverick

Genre: Family
          The Little Mermaid
                      Crater
Puss in Boots: The Last Wish
           Peter Pan & Wendy
                     Mummies

Genre: Fantasy
                                The Little Mermaid
                           Shazam! Fury of the Gods
                     Puss in Boots: The Last Wish
Sword Art Online the Movie -Progressive- Scherz...
                                  Peter Pan & Wendy

Genre: History
                       The Woman King
                       Hidden Figures
Prizefighter: The Life of Jem Belcher
                Ip Man: The Awakening
                             Medieval
```

```
Genre: Horror
     The Black Demon
      Evil Dead Rise
 The Pope's Exorcist
Project Wolf Hunting
           Scream VI

Genre: Music
Guillermo del Toro's Pinocchio
         Lyle, Lyle, Crocodile
                         Coco
     Blue's Big City Adventure
       Monster High: The Movie

Genre: Mystery
         Hypnotic
Operation Seawolf
        Scream VI
             Hunt
    Assassin Club

Genre: Romance
The Little Mermaid
           Ghosted
         Elemental
   Shotgun Wedding
        Love Again

Genre: Science Fiction
Spider-Man: Across the Spider-Verse
                           Hypnotic
   Transformers: Rise of the Beasts
           Avatar: The Way of Water
                  Operation Seawolf

Genre: TV Movie
                                Love's Greek to Me
Miraculous World: Shanghai – The Legend of Lady...
                             Teen Wolf: The Movie
                           Monster High: The Movie
                                        Megalodon

Genre: Thriller
              Fast X
John Wick: Chapter 4
            Hypnotic
     The Black Demon
      Evil Dead Rise
```

```
Genre: War
         Operation Seawolf
                      Sisu
Guy Ritchie's The Covenant
                    Ambush
   Sniper: The White Raven

Genre: Western
          Two Sinners and a Mule
                Django Unchained
                       Tombstone
Spirit: Stallion of the Cimarron
                  Spirit Untamed
```

```
#Top 5 production companies with maximum movie count
companies_column = df['production_companies']

# Create an empty dictionary to store company names and their movie
counts
company_counts = {}

# Iterate over each row in the companies column
for companies_list in companies_column:
    companies = eval(companies_list) # Convert the string
representation of list to a list
    for company in companies:
        if company in company_counts:
            company_counts[company] += 1 # Increment the movie count
        else:
            company_counts[company] = 1 # Add the company with initial
movie count

sorted_companies = sorted(company_counts.items(), key=lambda x: x[1],
reverse=True)

top_5_companies = sorted_companies[:5]

for company, count in top_5_companies:
    print(company, ": ", count)
```

```
Warner Bros. Pictures :  488
Universal Pictures :  471
Paramount :  373
Columbia Pictures :  365
20th Century Fox :  348
```

```
#Most popular movie in every 5 years with their popularity

# Convert the release_date column to datetime
df['release_date'] = pd.to_datetime(df['release_date'])
```

```python
# Create a new column for the release half_decade
df['release_hd'] = (df['release_date'].dt.year // 5) * 5

popular_movies = df.groupby('release_hd').apply(lambda x:
x.nlargest(1, 'popularity')) # 1 is for no. of rows we

for i, r in popular_movies.iterrows():
    print(f"Year: {r['release_hd']}-{r['release_hd']+4}")
    print("Movie:", r['title'])
    print("Popularity:", r['popularity'])
    print("\n")
```

```
Year: 1915-1919
Movie: The Birth of a Nation
Popularity: 12.324


Year: 1920-1924
Movie: The Cabinet of Dr. Caligari
Popularity: 20.457


Year: 1925-1929
Movie: Metropolis
Popularity: 20.14


Year: 1930-1934
Movie: Baby Face
Popularity: 89.292


Year: 1935-1939
Movie: Snow White and the Seven Dwarfs
Popularity: 67.737


Year: 1940-1944
Movie: Bambi
Popularity: 47.829


Year: 1945-1949
Movie: It's a Wonderful Life
Popularity: 27.584


Year: 1950-1954
Movie: Cinderella
Popularity: 75.643
```

```
Year: 1955-1959
Movie: Lady and the Tramp
Popularity: 40.061


Year: 1960-1964
Movie: Psycho
Popularity: 41.707


Year: 1965-1969
Movie: The Jungle Book
Popularity: 53.03


Year: 1970-1974
Movie: The Godfather
Popularity: 110.377


Year: 1975-1979
Movie: Caligula
Popularity: 96.711


Year: 1980-1984
Movie: Raiders of the Lost Ark
Popularity: 66.598


Year: 1985-1989
Movie: The Little Mermaid
Popularity: 243.272


Year: 1990-1994
Movie: Blank Check
Popularity: 133.648


Year: 1995-1999
Movie: Titanic
Popularity: 118.122


Year: 2000-2004
Movie: Eight Legged Freaks
Popularity: 346.156
```

```
Year: 2005-2009
Movie: Transformers: Beginnings
Popularity: 266.642


Year: 2010-2014
Movie: Transformers: Age of Extinction
Popularity: 424.422


Year: 2015-2019
Movie: Spider-Man: Into the Spider-Verse
Popularity: 914.969


Year: 2020-2024
Movie: Fast X
Popularity: 8363.473



C:\Users\Hajir\AppData\Local\Temp\ipykernel_12720\2184640187.py:9:
FutureWarning: DataFrameGroupBy.apply operated on the grouping
columns. This behavior is deprecated, and in a future version of
pandas the grouping columns will be excluded from the operation.
Either pass `include_groups=False` to exclude the groupings or
explicitly select the grouping columns after groupby to silence this
warning.
  popular_movies = df.groupby('release_hd').apply(lambda x:
x.nlargest(1, 'popularity')) # 1 is for no. of rows we
```